

UNIT-1: Introduction to Data Structures

1.1 Understand Various Types of Data Structures

1.1.1 Define Data Structure and Classify Them

Definition:

A **Data Structure** is a way of organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently.

Classification of Data Structures:

Data Structures are mainly classified into **two types**:

1. Linear Data Structures

- Elements are arranged in a sequential order.
- Each element is connected to the next element.

Examples:

- Array
- Linked List
- Stack
- Queue

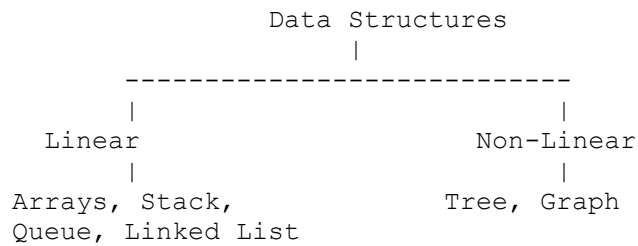
2. Non-Linear Data Structures

- Elements are arranged in a hierarchical or network form.
- One element can be connected to multiple elements.

Examples:

- Tree
- Graph

Classification Diagram:



1.1.2 Explain Linear Data Structures

Definition:

A **Linear Data Structure** is one in which data elements are stored one after another in a sequence.

Characteristics:

- Sequential arrangement.
- Easy to traverse.
- Each element has only one predecessor and one successor (except first and last).

Examples:

- Array
- Linked List
- Stack
- Queue

Advantages:

- Easy to implement.
- Faster traversal.
- Less memory complexity.

Disadvantages:

- Limited flexibility.
 - Insertion and deletion may take more time (especially in arrays).
-

1.1.3 Describe Non-Linear Data Structures

Definition:

A **Non-Linear Data Structure** is one in which data elements are not arranged sequentially.

Characteristics:

- Hierarchical or network structure.
- One node can have multiple connections.
- Suitable for representing complex relationships.

Examples:

- Tree
- Graph

Advantages:

- Efficient for representing hierarchical data.
- Faster searching in many cases.
- Better memory utilization for complex applications.

Disadvantages:

- More difficult to implement.
 - Traversal algorithms are more complex.
-

1.1.4 Explain Data Types and Abstract Data Types

Data Type

Definition:

A **Data Type** specifies the type of data a variable can store.

Types of Data Types

1. Primitive Data Types

- int
- float
- char
- double

2. Non-Primitive Data Types

- Array
- Structure
- Union
- Pointer

Abstract Data Type (ADT)

Definition:

An **Abstract Data Type (ADT)** is a logical description of a data structure that defines **what operations can be performed** without describing **how they are implemented**.

Examples:

- Stack (Push, Pop)
- Queue (Insert, Delete)
- List
- Tree

Advantages of ADT

- Data hiding
- Easy to modify
- Reusable code
- Better program design

Difference Between Data Type and ADT

Data Type	Abstract Data Type (ADT)
Defines the type of data	Defines operations on data
Built into programming language	User-defined concept
Example: int, char	Example: Stack, Queue

1.1.5 Explain Space and Time Complexities

Time Complexity

Definition:

Time Complexity is the amount of **time taken by an algorithm** to execute as the input size increases.

Common Time Complexities

Complexity	Meaning
$O(1)$	Constant Time
$O(\log n)$	Logarithmic Time
$O(n)$	Linear Time
$O(n \log n)$	Linear Logarithmic Time
$O(n^2)$	Quadratic Time

Example

Searching an element in an array:

- Linear Search $\rightarrow O(n)$
 - Binary Search $\rightarrow O(\log n)$
-

Space Complexity

Definition:

Space Complexity is the amount of **memory required** by an algorithm during execution.

Example

- A program using only one variable $\rightarrow O(1)$
 - A program storing an array of n elements $\rightarrow O(n)$
-

Importance of Time and Space Complexity

- Helps compare algorithms.
 - Selects the best algorithm.
 - Improves program performance.
 - Reduces execution time and memory usage.
-

1.2 Sequential Storage Representation

1.2.1 Various Sorting Techniques

1.2.1.1 Define Sorting

Definition:

Sorting is the process of arranging data elements in a specific order, either **ascending** (smallest to largest) or **descending** (largest to smallest).

Example:

Before Sorting:

45 12 30 18 25

After Ascending Sort:

12 18 25 30 45

After Descending Sort:

45 30 25 18 12

1.2.1.2 State the Need of Sorting

Need of Sorting

1. Makes searching faster.
 2. Organizes data efficiently.
 3. Reduces processing time.
 4. Simplifies data analysis.
 5. Helps in data management.
 6. Improves algorithm performance.
-

1.2.1.3 List the Methods of Sorting

Sorting Methods

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort
6. Heap Sort
7. Shell Sort
8. Radix Sort

In your syllabus, mainly focus on:

- Bubble Sort
- Selection Sort
- Insertion Sort
- Quick Sort
- Merge Sort

1.2.1.4 Bubble Sort

Sorting Technique

Bubble Sort repeatedly compares **adjacent elements** and swaps them if they are in the wrong order. After every pass, the largest element moves to the end.

Example:

25 15 40 10

Pass 1

15 25 40 10

15 25 10 40

Pass 2

15 10 25 40

Pass 3

10 15 25 40

Algorithm

Step 1: Read n elements.

Step 2: Repeat n-1 passes.

Step 3: Compare adjacent elements.

Step 4: Swap if left element > right element.

Step 5: Repeat until sorted.

Time Complexity

Case Complexity

Best $O(n)$

Average $O(n^2)$

Worst $O(n^2)$

C Program

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a[50], n, i, j, temp;
```

```
    printf("Enter number of elements: ");
```



```

scanf("%d",&n);

printf("Enter elements:\n");
for(i=0;i<n;i++)
    scanf("%d",&a[i]);

for(i=0;i<n-1;i++)
{
    for(j=0;j<n-i-1;j++)
    {
        if(a[j]>a[j+1])
        {
            temp=a[j];
            a[j]=a[j+1];
            a[j+1]=temp;
        }
    }
}

printf("Sorted array:\n");
for(i=0;i<n;i++)
    printf("%d ",a[i]);

return 0;
}

```

Selection Sort

Sorting Technique

Selection Sort finds the **smallest element** and places it at the correct position.

Example:

64 25 12 22

↓

12 25 64 22

↓

12 22 64 25

↓

12 22 25 64

Algorithm

Step 1: Select first element.

Step 2: Find smallest element.

Step 3: Swap with first position.
Step 4: Repeat for remaining elements.

Time Complexity

Case Complexity

Best $O(n^2)$

Average $O(n^2)$

Worst $O(n^2)$

C Program

```
#include<stdio.h>

int main()
{
    int a[50], n, i, j, min, temp;

    scanf("%d", &n);

    for(i=0; i<n; i++)
        scanf("%d", &a[i]);

    for(i=0; i<n-1; i++)
    {
        min=i;
        for(j=i+1; j<n; j++)
        {
            if(a[j]<a[min])
                min=j;
        }

        temp=a[i];
        a[i]=a[min];
        a[min]=temp;
    }

    for(i=0; i<n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Insertion Sort

Sorting Technique

Insertion Sort inserts each element into its proper position in the already sorted part of the array.

Example:

5 3 4 1

↓

3 5 4 1

↓

3 4 5 1

↓

1 3 4 5

Algorithm

Step 1: Assume first element is sorted.
Step 2: Pick next element.
Step 3: Compare with previous elements.
Step 4: Insert in correct position.
Step 5: Repeat until all elements are sorted.

Time Complexity

Case Complexity

Best $O(n)$

Average $O(n^2)$

Worst $O(n^2)$

C Program

```
#include<stdio.h>

int main()
{
    int a[50], n, i, j, key;

    scanf("%d", &n);

    for(i=0; i<n; i++)
```

```
        scanf("%d",&a[i]);

for(i=1;i<n;i++)
{
    key=a[i];
    j=i-1;

    while(j>=0 && a[j]>key)
    {
        a[j+1]=a[j];
        j--;
    }

    a[j+1]=key;
}

for(i=0;i<n;i++)
    printf("%d ",a[i]);

return 0;
}
```

Quick Sort

Sorting Technique

Quick Sort selects a **pivot element**, partitions the array into smaller and larger elements, and recursively sorts both parts.

Example:

38 27 43 10

Pivot = 38

↓

27 10 | 38 | 43

Algorithm

- Step 1: Select pivot.
- Step 2: Partition array.
- Step 3: Place pivot correctly.
- Step 4: Recursively sort left part.
- Step 5: Recursively sort right part.

Time Complexity

Case Complexity

Best $O(n \log n)$

Average $O(n \log n)$

Worst $O(n^2)$

Simple C Program

```
#include<stdio.h>

void quicksort(int a[], int low, int high);
int partition(int a[], int low, int high);

int main()
{
    int a[50], n, i;

    scanf("%d", &n);

    for(i=0; i<n; i++)
        scanf("%d", &a[i]);

    quicksort(a, 0, n-1);

    for(i=0; i<n; i++)
        printf("%d ", a[i]);

    return 0;
}

void quicksort(int a[], int low, int high)
{
    if(low<high)
    {
        int p=partition(a, low, high);
        quicksort(a, low, p-1);
        quicksort(a, p+1, high);
    }
}

int partition(int a[], int low, int high)
{
    int pivot=a[high];
    int i=low-1, j, temp;

    for(j=low; j<high; j++)
    {
        if(a[j]<pivot)
        {
            i++;
```

```
        temp=a[i];
        a[i]=a[j];
        a[j]=temp;
    }

    temp=a[i+1];
    a[i+1]=a[high];
    a[high]=temp;

    return i+1;
}
```

Merge Sort

Sorting Technique

Merge Sort divides the array into smaller parts, sorts them separately, and then merges them into one sorted array.

Example:

38 27 43 10

↓

38 27 | 43 10

↓

27 38 | 10 43

↓

10 27 38 43

Algorithm

Step 1: Divide array into two halves.
Step 2: Continue until one element remains.
Step 3: Merge sorted halves.
Step 4: Repeat until fully sorted.

Time Complexity

Case Complexity

Best $O(n \log n)$

Case Complexity

Average $O(n \log n)$

Worst $O(n \log n)$

Simple C Program

```
/* Merge Sort uses Divide and Conquer.  
Main functions:  
1. Divide the array.  
2. Merge sorted arrays.  
*/
```

(In many diploma exams, only the algorithm and complexity of Merge Sort are asked. If a program is required, you can write the standard merge sort implementation.)

Comparison of Sorting Techniques

Sorting Method	Best	Average	Worst
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

1.2.2 Various Searching Techniques

1.2.2.1 Define Searching

Definition:

Searching is the process of finding the location of a required element (key) in a collection of data.

If the element is found, its position is returned; otherwise, it reports that the element is not present.

Example:

Array: 10 20 30 40 50

Search Key = 30

Output: Element found at Position 3

1.2.2.2 State the Need of Searching

Need of Searching

1. To locate a specific element quickly.
 2. To retrieve required information efficiently.
 3. To reduce searching time.
 4. To improve program performance.
 5. Used in databases, file systems, and applications.
-

1.2.2.3 List Two Types of Searching

Types of Searching

1. **Linear Search (Sequential Search)**
 2. **Binary Search**
-

1.2.2.4 Linear Search

1. Searching Technique

Linear Search checks each element one by one until the required element is found or the list ends.

Example:

Array : 25 18 40 35 50

Search = 35

25 X

18 X

40 X

35 ✓ Found

2. Algorithm

Step 1: Read the array and key element.

Step 2: Start from the first element.

Step 3: Compare each element with the key.

Step 4: If matched, display its position.

Step 5: Otherwise continue.

Step 6: If end of array is reached, display "Element not found".

3. Time Complexity

Case Complexity

Best $O(1)$

Average $O(n)$

Worst $O(n)$

4. C Program (Linear Search)

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a[50], n, key, i, found=0;
```

```
printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter elements:\n");
for(i=0;i<n;i++)
    scanf("%d",&a[i]);

printf("Enter element to search: ");
scanf("%d",&key);

for(i=0;i<n;i++)
{
    if(a[i]==key)
    {
        printf("Element found at position %d",i+1);
        found=1;
        break;
    }
}

if(found==0)
    printf("Element not found");

return 0;
}
```

Binary Search

Definition

Binary Search searches an element by repeatedly dividing the sorted array into two halves.

Note: Binary Search works **only on a sorted array**.

1. Searching Technique

Example:

Array : 10 20 30 40 50 60 70

Search = 50

Middle = 40

50 > 40

Search right half

Middle = 60

50 < 60

Search left half

50 Found

2. Binary Search Algorithm (Without Recursion)

Step 1: Read sorted array.

Step 2: Set low = 0 and high = n-1.

Step 3: Find middle = (low + high)/2.

Step 4: If key == middle element, display position.

Step 5: If key < middle element, set high = middle-1.

Step 6: Else set low = middle+1.

Step 7: Repeat until found or low > high.

Time Complexity

Case Complexity

Best $O(1)$

Average $O(\log n)$

Worst $O(\log n)$

C Program (Binary Search Without Recursion)

```
#include<stdio.h>

int main()
{
    int a[50], n, key;
    int low, high, mid, found=0, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter sorted elements:\n");
    for(i=0; i<n; i++)
        scanf("%d", &a[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);
```

```
low=0;
high=n-1;

while(low<=high)
{
    mid=(low+high)/2;

    if(a[mid]==key)
    {
        printf("Element found at position %d",mid+1);
        found=1;
        break;
    }
    else if(key<a[mid])
        high=mid-1;
    else
        low=mid+1;
}

if(found==0)
    printf("Element not found");

return 0;
}
```

Binary Search with Recursion

Searching Technique

The array is divided into two halves recursively until the element is found or the search interval becomes empty.

Algorithm

- Step 1: Find middle element.
 - Step 2: If key == middle element, return position.
 - Step 3: If key < middle element, search left half recursively.
 - Step 4: Otherwise search right half recursively.
 - Step 5: Stop when element is found or low > high.
-

Time Complexity

Case Complexity

Best $O(1)$

Average $O(\log n)$

Worst $O(\log n)$

C Program (Binary Search Using Recursion)

```
#include<stdio.h>

int binarySearch(int a[], int low, int high, int key)
{
    if(low>high)
        return -1;

    int mid=(low+high)/2;

    if(a[mid]==key)
        return mid;
    else if(key<a[mid])
        return binarySearch(a,low,mid-1,key);
    else
        return binarySearch(a,mid+1,high,key);
}

int main()
{
    int a[50], n, key, i, pos;

    printf("Enter number of elements: ");
    scanf("%d",&n);

    printf("Enter sorted elements:\n");
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);

    printf("Enter element to search: ");
    scanf("%d",&key);

    pos=binarySearch(a,0,n-1,key);

    if(pos==-1)
        printf("Element not found");
    else
        printf("Element found at position %d",pos+1);

    return 0;
}
```

Difference Between Linear Search and Binary Search

Linear Search

Works on sorted and unsorted data

Checks elements one by one

Time Complexity: $O(n)$

Simple to implement

Binary Search

Works only on sorted data

Divides array into two halves

Time Complexity: $O(\log n)$

Faster but requires sorted data

UNIT-2: Linked Storage Representation – Linked Lists

2.1 List the Advantages & Disadvantages of Linked Lists over Arrays

Advantages of Linked List over Array

1. Dynamic size (can grow or shrink at runtime).
2. Easy insertion and deletion of elements.
3. No need for contiguous memory.
4. Better memory utilization.
5. Suitable for implementing stacks, queues, and graphs.

Disadvantages of Linked List

1. Extra memory is required for pointers.
 2. Sequential access only (no direct indexing).
 3. Searching is slower than arrays.
 4. More complex implementation.
 5. Pointer handling may lead to errors.
-

2.2 State the Purpose of Dummy Header

Definition

A **Dummy Header** is an extra node placed at the beginning of a linked list. It does not store actual data.

Purpose

- Simplifies insertion and deletion.
- Eliminates special cases for the first node.

- Makes programming easier.
 - Improves code readability.
-

2.3 Explain Singly Linked List and Doubly Linked List

A. Singly Linked List (SLL)

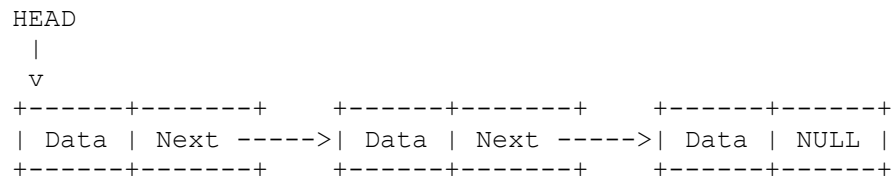
2.3.1 Structure

Definition

A **Singly Linked List** is a collection of nodes where each node contains:

- Data
- Pointer to the next node

Diagram



2.3.2 Creation

Steps

1. Create a new node.
 2. Store data.
 3. Set next pointer to NULL.
 4. If list is empty, make it HEAD.
 5. Otherwise attach at the end.
-

2.3.3 Representation of Node using C Structure

```
struct node
{
    int data;
    struct node *next;
};
```

2.3.4 Operations

Insertion

Algorithm

Step 1: Create new node.
Step 2: Store data.
Step 3: Set next pointer.
Step 4: Update links.

Deletion

Algorithm

Step 1: Find node.
Step 2: Change previous node's next pointer.
Step 3: Free deleted node.

Traversal

Algorithm

Step 1: Start from HEAD.
Step 2: Visit each node.
Step 3: Move to next node.
Step 4: Stop at NULL.

Sorting

Algorithm

Step 1: Compare adjacent nodes.
Step 2: Swap data if required.
Step 3: Repeat until sorted.

2.3.5 Search and Replace

Searching

Step 1: Start from HEAD.
Step 2: Compare each node with key.
Step 3: If found, display position.
Step 4: Otherwise continue until NULL.

Replacing

Step 1: Search the required node.
Step 2: Replace old value with new value.
Step 3: Display updated list.

2.3.6 C Program (All Operations – Singly Linked List)

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *head=NULL,*temp,*newnode;

void create()
{
    int choice=1;

    while(choice)
    {
        newnode=(struct node*)malloc(sizeof(struct node));

        printf("Enter data: ");
        scanf("%d",&newnode->data);

        newnode->next=NULL;

        if(head==NULL)
        {
            head=temp=newnode;
        }
        else
        {
            temp->next=newnode;
            temp=newnode;
        }

        printf("Continue (1/0): ");
        scanf("%d",&choice);
    }
}
```

```

    }
}

void display()
{
    temp=head;

    while(temp!=NULL)
    {
        printf("%d -> ",temp->data);
        temp=temp->next;
    }

    printf("NULL\n");
}

int main()
{
    create();
    display();

    return 0;
}

```

(Insertion, deletion, searching, sorting, and replacing functions can be added similarly in exams if asked separately.)

B. Doubly Linked List (DLL)

2.3.1 Structure

Definition

A **Doubly Linked List** is a linked list where each node contains:

- Previous pointer
- Data
- Next pointer

Diagram

```

NULL <-+-----+-----+-----+<->+-----+-----+-----+<->+-----+-----+---
-----+> NULL
      | Prev | Data | Next |      | Prev | Data | Next |      | Prev | Data |
Next | +-----+-----+-----+  +-----+-----+-----+  +-----+-----+---
-----+

```

2.3.2 Creation

Steps

1. Create node.
 2. Store data.
 3. Set previous and next pointers.
 4. Link nodes properly.
-

2.3.3 Representation using C Structure

```
struct node
{
    int data;
    struct node *prev;
    struct node *next;
};
```

2.3.4 Operations

Insertion

- Beginning
- End
- Middle

Deletion

- Beginning
- End
- Specific node

Traversal

- Forward Traversal
- Backward Traversal

Sorting

- Compare nodes.
- Swap data.

- Repeat until sorted.

2.3.5 Search and Replace

Searching

Step 1: Start from HEAD.
Step 2: Compare each node.
Step 3: If found display position.
Step 4: Otherwise continue.

Replacing

Step 1: Search node.
Step 2: Replace data.
Step 3: Display updated list.

2.3.6 C Program (Basic Doubly Linked List)

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
    int data;
    struct node *prev,*next;
};

int main()
{
    struct node *head,*second;

    head=(struct node*)malloc(sizeof(struct node));
    second=(struct node*)malloc(sizeof(struct node));

    head->data=10;
    head->prev=NULL;
    head->next=second;

    second->data=20;
    second->prev=head;
    second->next=NULL;

    printf("%d %d",head->data,second->data);

    return 0;
}
```

Difference Between Singly and Doubly Linked List

Singly Linked List	Doubly Linked List
One pointer (next)	Two pointers (prev, next)
Forward traversal only	Forward and backward traversal
Less memory	More memory
Simple implementation	More complex implementation
Faster due to less memory	Easier insertion/deletion

2.4 Singly Circular Linked List (SCLL)

2.4.1 Definition

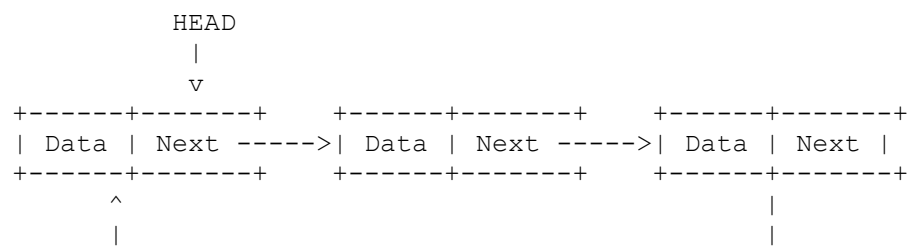
Definition

A **Singly Circular Linked List (SCLL)** is a linked list in which the **last node points back to the first node (HEAD)** instead of pointing to `NULL`.

Thus, the list forms a **circle**.

2.4.2 Structure

Diagram



Node Representation in C

```
struct node
{
    int data;
    struct node *next;
};
```

2.4.3 Advantages of a Singly Circular Linked List over a Singly Linked List

Advantages

1. Last node points back to the first node.
 2. Traversal can start from any node.
 3. No NULL pointer at the end.
 4. Suitable for circular applications like CPU scheduling.
 5. Easier implementation of circular queues.
 6. Continuous traversal is possible.
-

2.4.4 Changes Required in a Singly Linked List Program to Make it a Singly Circular Linked List

Required Changes

1. Replace the last node's `next = NULL` with:
 2. `last->next = head;`
 3. During traversal, stop when the pointer reaches **HEAD** again instead of `NULL`.
 4. While inserting the last node:
 5. `newnode->next = head;`
 6. During deletion, update the last node's next pointer to the new head.
-

2.5 Doubly Circular Linked List (DCLL)

2.5.1 Definition

Definition

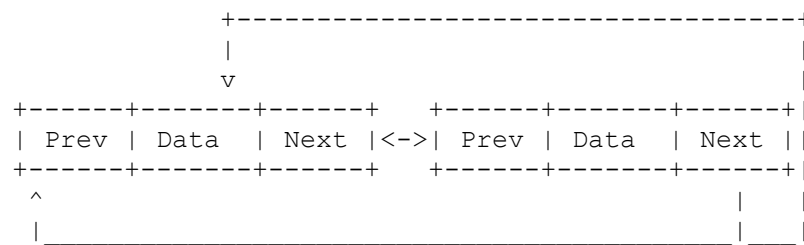
A **Doubly Circular Linked List (DCLL)** is a linked list in which:

- Every node has **two pointers** (`prev` and `next`).
- The **last node points to the first node**.
- The **first node points back to the last node** through the `prev` pointer.

There is **no NULL pointer** in the list.

2.5.2 Structure

Diagram



Node Representation in C

```
struct node
{
    int data;
    struct node *prev;
    struct node *next;
};
```

2.5.3 Advantages of a Doubly Circular Linked List over a Doubly Linked List

Advantages

1. Supports forward and backward traversal.
2. No NULL pointers at either end.
3. Traversal can start from any node.

4. Easier insertion and deletion at both ends.
 5. Suitable for music playlists, browser history, and round-robin scheduling.
 6. Continuous traversal in both directions.
-

2.5.4 Changes Required in a Doubly Linked List Program to Make it a Doubly Circular Linked List

Required Changes

1. Instead of:
2. `last->next = NULL;`

write:

```
last->next = head;
```

3. Instead of:
4. `head->prev = NULL;`

write:

```
head->prev = last;
```

5. During insertion at the end:
 6. `newnode->next = head;`
 7. `newnode->prev = last;`
 8. `last->next = newnode;`
 9. `head->prev = newnode;`
 10. During traversal:
 - o Stop when the pointer reaches **HEAD** again.
 - o Do **not** check for `NULL`.
-

Difference Between SCLL and DCLL

Singly Circular Linked List

Doubly Circular Linked List

One pointer (`next`)

Two pointers (`prev`, `next`)

Forward traversal only

Forward and backward traversal

Singly Circular Linked List**Doubly Circular Linked List**

Less memory required

More memory required

Simple implementation

More complex implementation

Last node points to head

Last node points to head and head points to last

UNIT-3: Linear Data Structures – Stacks

3.1 Define Stack

Definition

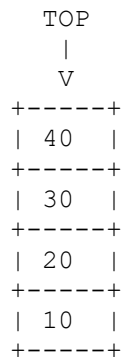
A **Stack** is a **linear data structure** in which insertion and deletion of elements are performed at **one end only**, called the **TOP**.

A stack follows the **LIFO (Last In First Out)** principle.

Example:

- Stack of plates.
- Books stacked one above another.

Diagram



3.2 Explain the Push, Pop and Display Operations of a Stack

1. Push Operation

Definition

Push operation inserts a new element at the **TOP** of the stack.

Algorithm

Step 1: Check whether stack is full.
Step 2: If full, display "Stack Overflow".
Step 3: Otherwise increment TOP.
Step 4: Insert the element.

2. Pop Operation

Definition

Pop operation removes the **topmost element** from the stack.

Algorithm

Step 1: Check whether stack is empty.
Step 2: If empty, display "Stack Underflow".
Step 3: Otherwise delete the top element.
Step 4: Decrement TOP.

3. Display Operation

Definition

Display operation prints all stack elements from **TOP to Bottom**.

Algorithm

Step 1: Check whether stack is empty.
Step 2: If empty, display "Stack Empty".
Step 3: Otherwise print elements from TOP to 0.

3.3 Explain Array Implementation of a Stack with Various Operations

Definition

In this implementation, the stack is stored using an **array**.

Declaration

```
int stack[100];  
int top = -1;
```

Operations

Push

- Increase `top`
- Insert element

Pop

- Delete element at `top`
- Decrease `top`

Peek

- Display top element.

Display

- Print all elements from top to bottom.

Diagram

Stack[5]

```
TOP
|
V
+-----+
| 40  |
+-----+
| 30  |
+-----+
| 20  |
+-----+
| 10  |
+-----+
```

Advantages

- Easy to implement.
- Faster access.

Disadvantages

- Fixed size.
- Stack overflow may occur.

3.4 Explain the Program for Array Implementation of a Stack with Various Operations

C Program

```
#include<stdio.h>

#define MAX 5

int stack[MAX];
int top=-1;

void push()
{
    int x;

    if(top==MAX-1)
        printf("Stack Overflow\n");
    else
    {
        printf("Enter element: ");
        scanf("%d",&x);

        top++;
        stack[top]=x;
    }
}

void pop()
{
    if(top== -1)
        printf("Stack Underflow\n");
    else
    {
        printf("Deleted element = %d\n",stack[top]);
        top--;
    }
}

void display()
{
    int i;

    if(top== -1)
        printf("Stack Empty\n");
    else
    {
        printf("Stack Elements:\n");

        for(i=top;i>=0;i--)
            printf("%d\n",stack[i]);
    }
}
```

```

    }
}

int main()
{
    push();
    push();
    display();
    pop();
    display();

    return 0;
}

```

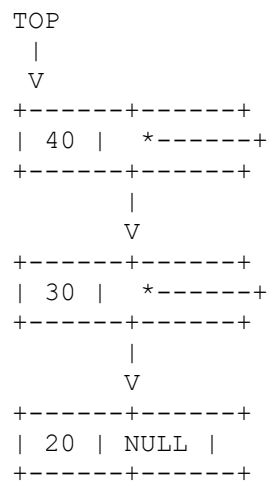
3.5 Explain Linked List Implementation of a Stack with Various Operations

Definition

A stack can also be implemented using a **Linked List**.

The **HEAD** acts as the **TOP** of the stack.

Diagram



Operations

Push

- Create new node.
- Insert at beginning.

Pop

- Delete first node.

Display

- Traverse and print all nodes.

Advantages

- Dynamic size.
- No stack overflow until memory is full.
- Efficient insertion and deletion.

Disadvantages

- Extra memory for pointer.
 - Slightly slower than arrays.
-

3.6 Explain the Program for Linked List Implementation of a Stack with Various Operations

C Program

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *top=NULL;

void push()
{
    struct node *newnode;

    newnode=(struct node*)malloc(sizeof(struct node));

    printf("Enter data: ");
    scanf("%d",&newnode->data);
```



```
        newnode->next=top;
        top=newnode;
    }

void pop()
{
    struct node *temp;

    if(top==NULL)
    {
        printf("Stack Underflow\n");
        return;
    }

    temp=top;

    printf("Deleted = %d\n",top->data);

    top=top->next;

    free(temp);
}

void display()
{
    struct node *temp=top;

    if(top==NULL)
    {
        printf("Stack Empty\n");
        return;
    }

    while(temp!=NULL)
    {
        printf("%d\n",temp->data);
        temp=temp->next;
    }
}

int main()
{
    push();
    push();
    display();
    pop();
    display();

    return 0;
}
```

Difference Between Array Stack and Linked List Stack

Array Stack	Linked List Stack
Uses array	Uses linked list
Fixed size	Dynamic size
Fast access	Slightly slower
May cause overflow	Overflow only when memory is full
No extra pointer memory	Extra memory needed for pointers

Applications of Stack

1. Function calls (Recursion).
 2. Expression evaluation.
 3. Parentheses matching.
 4. Undo/Redo operations.
 5. Browser history.
 6. Backtracking algorithms.
-

3.7 List the Applications of Stacks

Applications of Stack

1. Function calls and recursion.
2. Expression evaluation.
3. Infix to Postfix conversion.
4. Postfix expression evaluation.
5. Parentheses matching.
6. Undo and Redo operations in editors.
7. Browser back/forward navigation.
8. Backtracking algorithms (Maze, N-Queens).
9. Syntax checking in compilers.
10. Depth First Search (DFS) in graphs.

3.8 Convert Infix Expression to Postfix Expression

Definition

- **Infix Expression:** Operator is placed **between operands**.
 - Example: $A + B$
- **Postfix Expression:** Operator is placed **after operands**.
 - Example: $AB+$

Need for Conversion

- Easier for computers to evaluate.
- No need for brackets.
- Faster expression evaluation.

Algorithm for Infix to Postfix Conversion

Step 1: Create an empty stack.
Step 2: Scan the infix expression from left to right.
Step 3: If operand, add it to postfix.
Step 4: If '(', push into stack.
Step 5: If ')', pop until '(' is found.
Step 6: If operator, pop higher or equal precedence operators and add to postfix.
Step 7: Push the current operator.
Step 8: Repeat until expression ends.
Step 9: Pop remaining operators.
Step 10: Display postfix expression.

Operator Precedence

Operator Priority

()	Highest
^	Highest
* / %	High
+ -	Low

Example

Infix

A+B*C

Conversion

Scan A → Output = A

Scan + → Push +

Scan B → Output = AB

Scan * → Push *

Scan C → Output = ABC

Pop *

Output = ABC*

Pop +

Output = ABC*+

Final Postfix

ABC*+

3.9 Explain the Program for Conversion of Infix Expression to Postfix Expression

C Program

```
#include<stdio.h>
#include<ctype.h>

char stack[100];
int top=-1;

void push(char x)
{
    stack[++top]=x;
}

char pop()
{
    return stack[top--];
}
```

```

}

int priority(char x)
{
    if(x=='(')
        return 0;
    if(x=='+'||x=='-')
        return 1;
    if(x=='*'||x=='/')
        return 2;
    return 3;
}

int main()
{
    char exp[100],ch;
    int i=0;

    printf("Enter Infix Expression: ");
    scanf("%s",exp);

    while(exp[i]!='\0')
    {
        ch=exp[i];

        if(isalnum(ch))
            printf("%c",ch);

        else if(ch=='(')
            push(ch);

        else if(ch==')')
        {
            while(stack[top]!='(')
                printf("%c",pop());

            pop();
        }

        else
        {
            while(top!=-1 && priority(stack[top])>=priority(ch))
                printf("%c",pop());

            push(ch);
        }

        i++;
    }

    while(top!=-1)
        printf("%c",pop());

    return 0;
}

```

3.10 Evaluate Postfix Expression

Definition

Postfix Evaluation is the process of finding the result of a postfix expression using a **Stack**.

Algorithm

Step 1: Create an empty stack.
Step 2: Scan postfix expression from left to right.
Step 3: If operand, push it into stack.
Step 4: If operator,
 Pop two operands.
 Perform operation.
 Push result back.
Step 5: Repeat until expression ends.
Step 6: Final value in stack is the answer.

Example

Postfix Expression

23*54*+

Evaluation

2 Push

3 Push

* $\rightarrow 2 \times 3 = 6$

Push 5

Push 4

* $\rightarrow 5 \times 4 = 20$

+ $\rightarrow 6 + 20 = 26$

Result

26

3.11 Explain the Program for Evaluating Postfix Expression

C Program

```
#include<stdio.h>
#include<ctype.h>

int stack[100];
int top=-1;

void push(int x)
{
    stack[++top]=x;
}

int pop()
{
    return stack[top--];
}

int main()
{
    char exp[100];
    int i=0,a,b;

    printf("Enter Postfix Expression: ");
    scanf("%s",exp);

    while(exp[i]!='\0')
    {
        if(isdigit(exp[i]))
        {
            push(exp[i]-48);
        }
        else
        {
            b=pop();
            a=pop();

            switch(exp[i])
            {
                case '+': push(a+b); break;
                case '-': push(a-b); break;
                case '*': push(a*b); break;
                case '/': push(a/b); break;
            }
        }

        i++;
    }

    printf("Result = %d",pop());

    return 0;
}
```

Difference Between Infix and Postfix Expressions

Infix	Postfix
Operator between operands	Operator after operands
Uses brackets	No brackets required
Harder for computers to evaluate	Easier for computers to evaluate
Example: $A+B$	Example: $AB+$

Important Exam Points

- **Stack Applications:** Recursion, expression evaluation, infix-to-postfix conversion, postfix evaluation, browser history, undo/redo.
- **Infix Expression:** Operator is between operands (e.g., $A+B$).
- **Postfix Expression:** Operator comes after operands (e.g., $AB+$).
- **Infix to Postfix:** Uses a stack and operator precedence rules.
- **Postfix Evaluation:** Uses a stack by pushing operands and applying operators.
- **Advantages of Postfix:** No brackets needed and easier for computers to evaluate.

These answers are written in diploma exam style and are suitable for 2-mark, 5-mark, and 10-mark questions.

UNIT-4: Linear Data Structures – Queues

4.1 Define Queue

Definition

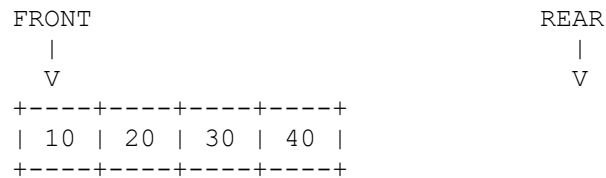
A **Queue** is a **linear data structure** in which insertion is performed at the **REAR** end and deletion is performed at the **FRONT** end.

A Queue follows the **FIFO (First In First Out)** principle.

Example:

- People standing in a ticket queue.
- Customers waiting in a bank.

Diagram



4.2 Explain the Insertion, Deletion and Display Operations on Queues

1. Insertion (Enqueue)

Definition

Insertion (Enqueue) adds an element at the **REAR** of the queue.

Algorithm

Step 1: Check whether the queue is full.
Step 2: If full, display "Queue Overflow".
Step 3: Otherwise increment REAR.

Step 4: Insert the new element.

2. Deletion (Dequeue)

Definition

Deletion (Dequeue) removes an element from the **FRONT** of the queue.

Algorithm

Step 1: Check whether the queue is empty.
Step 2: If empty, display "Queue Underflow".
Step 3: Otherwise delete the FRONT element.
Step 4: Increment FRONT.

3. Display

Definition

Display operation prints all elements from **FRONT** to **REAR**.

Algorithm

Step 1: Check whether queue is empty.
Step 2: If empty, display "Queue Empty".
Step 3: Otherwise display elements from FRONT to REAR.

4.3 Explain Array Implementation of a Queue with Various Operations

Definition

In Array implementation, Queue elements are stored in a **one-dimensional array**.

Declaration

```
int queue[100];  
int front = -1;  
int rear = -1;
```

Operations

Enqueue

- Insert at Rear.

Deque

- Delete from Front.

Peek

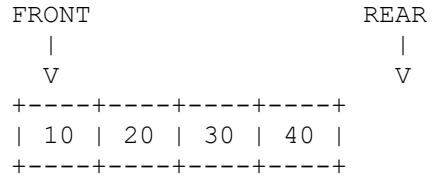
- Display Front element.

Display

- Print all elements.

Diagram

Queue [5]



Advantages

- Easy to implement.
- Fast access.
- Simple programming.

Disadvantages

- Fixed size.
- Memory wastage after deletions.
- Queue Overflow may occur even when free space exists.

4.4 Explain the Program for Array Implementation of a Queue with Various Operations

C Program

```
#include<stdio.h>

#define MAX 5

int queue[MAX];
int front=-1,rear=-1;

void enqueue()
{
    int x;

    if(rear==MAX-1)
    {
        printf("Queue Overflow\n");
        return;
    }

    printf("Enter element: ");
    scanf("%d",&x);

    if(front==-1)
        front=0;

    rear++;
    queue[rear]=x;
}

void dequeue()
{
    if(front==-1 || front>rear)
    {
        printf("Queue Underflow\n");
        return;
    }

    printf("Deleted = %d\n",queue[front]);
    front++;
}

void display()
{
    int i;

    if(front==-1 || front>rear)
    {
        printf("Queue Empty\n");
        return;
    }
}
```

```

        for(i=front;i<=rear;i++)
            printf("%d ",queue[i]);

        printf("\n");
    }

int main()
{
    enqueue();
    enqueue();
    display();
    dequeue();
    display();

    return 0;
}

```

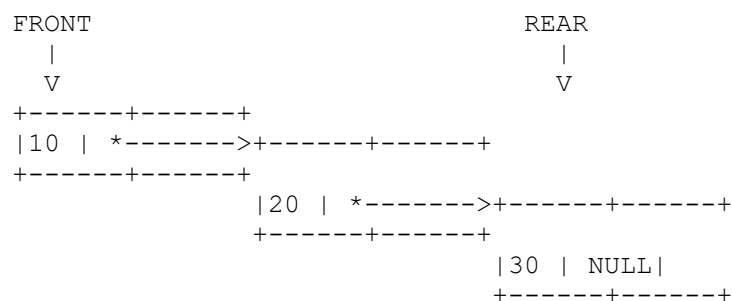
4.5 Explain Linked List Implementation of a Queue with Various Operations

Definition

A Queue can also be implemented using a **Linked List**.

- **Insertion** is performed at the **Rear**.
- **Deletion** is performed at the **Front**.

Diagram



Operations

Enqueue

- Create a new node.
- Insert at Rear.

Dequeue

- Delete node from Front.

Display

- Traverse from Front to Rear.

Advantages

- Dynamic size.
- No fixed memory limit.
- Efficient insertion and deletion.

Disadvantages

- Extra memory required for pointers.
 - Slightly slower than arrays.
-

4.6 Explain the Program for Linked List Implementation of a Queue with Various Operations

C Program

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *front=NULL,*rear=NULL;

void enqueue()
{
    struct node *newnode;

    newnode=(struct node*)malloc(sizeof(struct node));

    printf("Enter data: ");
    scanf("%d",&newnode->data);
```

```

newnode->next=NULL;

if (front==NULL)
{
    front=rear=newnode;
}
else
{
    rear->next=newnode;
    rear=newnode;
}
}

void dequeue()
{
    struct node *temp;

    if (front==NULL)
    {
        printf("Queue Underflow\n");
        return;
    }

    temp=front;

    printf("Deleted = %d\n",front->data);

    front=front->next;

    free(temp);

    if (front==NULL)
        rear=NULL;
}

void display()
{
    struct node *temp=front;

    if (front==NULL)
    {
        printf("Queue Empty\n");
        return;
    }

    while (temp!=NULL)
    {
        printf("%d ",temp->data);
        temp=temp->next;
    }

    printf("\n");
}

int main()
{
    enqueue();

```

```
enqueue();  
display();  
dequeue();  
display();  
  
return 0;  
}
```

Difference Between Stack and Queue

Stack	Queue
LIFO (Last In First Out)	FIFO (First In First Out)
Insertion and deletion at the same end (TOP)	Insertion at REAR and deletion at FRONT
Operations: Push, Pop	Operations: Enqueue, Dequeue
Example: Stack of plates	Example: Queue at a ticket counter

Applications of Queue

1. CPU Scheduling.
 2. Printer Spooling.
 3. Ticket Reservation Systems.
 4. Breadth First Search (BFS).
 5. Keyboard Input Buffer.
 6. Call Center Management.
 7. Network Packet Scheduling.
 8. Process Scheduling in Operating Systems.
-
-

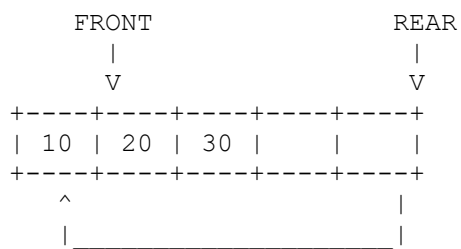
4.7 Know about Circular Queues

Definition

A **Circular Queue** is a type of queue in which the **last position is connected to the first position**, forming a circle.

It follows the **FIFO (First In First Out)** principle and efficiently utilizes memory by reusing empty spaces.

Diagram



Advantages

1. Efficient memory utilization.
2. Avoids memory wastage.
3. Reuses empty positions.
4. Suitable for CPU scheduling and buffering.

4.8 Explain Array Implementation of a Circular Queue with Various Operations

Definition

A Circular Queue is implemented using an array where **FRONT** and **REAR** move circularly using the modulo (%) operator.

Declaration

```
#define MAX 5
int queue[MAX];
int front = -1, rear = -1;
```

Operations

Enqueue (Insertion)

- Check whether the queue is full.
- Insert element at the rear.
- Update `rear = (rear + 1) % MAX`.

Dequeue (Deletion)

- Remove element from the front.
- Update `front = (front + 1) % MAX`.

Display

- Print all elements from front to rear in circular order.

Advantages

- No memory wastage.
 - Better than a simple queue.
 - Efficient utilization of array space.
-

4.9 Explain the Program for Array Implementation of a Circular Queue with Various Operations

C Program

```
#include<stdio.h>

#define MAX 5

int queue[MAX];
int front=-1, rear=-1;

void enqueue(int x)
{
    if((rear+1)%MAX==front)
    {
        printf("Queue Overflow\n");
        return;
    }
}
```

```

        if(front== -1)
            front=0;

        rear=(rear+1)%MAX;
        queue[rear]=x;
    }

void dequeue()
{
    if(front== -1)
    {
        printf("Queue Underflow\n");
        return;
    }

    printf("Deleted = %d\n",queue[front]);

    if(front==rear)
        front=rear=-1;
    else
        front=(front+1)%MAX;
}

void display()
{
    int i;

    if(front== -1)
    {
        printf("Queue Empty\n");
        return;
    }

    i=front;

    while(i!=rear)
    {
        printf("%d ",queue[i]);
        i=(i+1)%MAX;
    }

    printf("%d\n",queue[rear]);
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    display();

    dequeue();

    display();
}

```

```
    return 0;
}
```

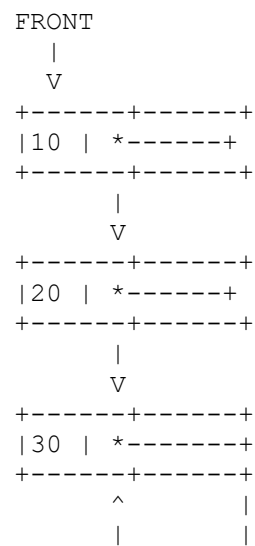
4.10 Explain Linked List Implementation of a Circular Queue with Various Operations

Definition

A Circular Queue can also be implemented using a **Circular Linked List**.

- The last node points to the first node.
- `Rear->next = Front.`

Diagram



Operations

Enqueue

- Create a new node.
- Insert at rear.
- Make rear point to front.

Dequeue

- Delete front node.

- Update front.
- Maintain circular link.

Display

- Traverse until the pointer reaches the front again.

Advantages

- Dynamic memory allocation.
- No fixed size.
- Efficient memory usage.

4.11 Explain the Program for Linked List Implementation of a Circular Queue with Various Operations

Simple C Program

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *front=NULL,*rear=NULL;

void enqueue(int x)
{
    struct node *newnode;

    newnode=(struct node*)malloc(sizeof(struct node));

    newnode->data=x;

    if(front==NULL)
    {
        front=rear=newnode;
        rear->next=front;
    }
    else
    {
        rear->next=newnode;
        rear=newnode;
    }
}
```

```

        rear->next=front;
    }
}

void display()
{
    struct node *temp;

    if(front==NULL)
    {
        printf("Queue Empty\n");
        return;
    }

    temp=front;

    do
    {
        printf("%d ",temp->data);
        temp=temp->next;
    }
    while(temp!=front);

    printf("\n");
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    display();

    return 0;
}

```

4.12 List the Applications of Queues

Applications

1. CPU Scheduling.
2. Printer Queue.
3. Ticket Reservation System.
4. Keyboard Buffer.
5. Breadth First Search (BFS).
6. Process Scheduling.
7. Call Center Management.
8. Network Packet Scheduling.
9. Disk Scheduling.

4.13 Know about Priority Queues

Definition

A **Priority Queue** is a special type of queue in which each element is assigned a **priority**.

Elements with **higher priority are deleted before lower-priority elements**, regardless of their insertion order.

Example

Priority	Element
1	A
2	B
3	C

Deletion Order:

C → B → A

(Highest priority is deleted first.)

Applications

1. CPU Process Scheduling.
2. Hospital Emergency Systems.
3. Dijkstra's Shortest Path Algorithm.
4. Printer Scheduling.
5. Network Routing.

Advantages

- Processes important tasks first.
- Efficient scheduling.
- Reduces waiting time for high-priority tasks.

Disadvantages

- More complex implementation.

- Lower-priority tasks may wait for a long time (starvation).

Difference Between Queue and Priority Queue

Queue	Priority Queue
Follows FIFO	Follows Priority Order
First inserted is deleted first	Highest priority is deleted first
All elements have equal priority	Every element has a priority
Simple implementation	More complex implementation

UNIT-5: Non-Linear Data Structures – Trees

5.1 Define a Tree

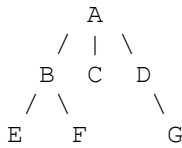
Definition

A **Tree** is a **non-linear data structure** that consists of a collection of **nodes** connected by **edges**.

It has one **Root node**, and every node (except the root) has exactly one parent.

Trees are used to represent **hierarchical data**.

Diagram



5.2 State the Importance of Various Terminologies Related to Tree

5.2.1 Tree Terminologies

1. Root

The topmost node of a tree.

Example: A

2. Edge

The connection between two nodes.

Example: $A \rightarrow B$

3. Parent

A node that has one or more child nodes.

Example: A is the parent of B, C, and D.

4. Child

A node connected below a parent.

Example: B is the child of A.

5. Siblings

Nodes having the same parent.

Example: B, C, and D are siblings.

6. Leaf Node (External Node)

A node that has no children.

Example: E, F, C, and G.

7. Internal Node

A node having at least one child.

Example: A, B, and D.

8. Degree of a Node

The number of children of a node.

Example: Degree of A = 3.

9. Level

The position of a node from the root.

- Root is at **Level 0**.
 - Its children are at **Level 1**.
-

10. Height

The number of edges in the longest path from a node to a leaf.

11. Depth

The number of edges from the root to a node.

12. Path

A sequence of nodes connected by edges.

Example:

A → B → E

13. Subtree

A tree formed by any node and all of its descendants.

Example:



14. Forest

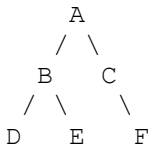
A collection of two or more disjoint trees.

5.3 Define Binary Tree

Definition

A **Binary Tree** is a tree in which **each node has at most two children**, called the **Left Child** and **Right Child**.

Diagram



Advantages

- Faster searching.
 - Easy insertion and deletion.
 - Efficient data organization.
-

5.4 Difference Between General Tree and Binary Tree

General Tree	Binary Tree
A node can have any number of children.	A node can have at most two children.
More complex structure.	Simpler structure.
Used for hierarchical data.	Used in searching and sorting.
No restriction on children.	Left and Right child only.

5.5 Conversion of General Tree to Binary Tree

Method

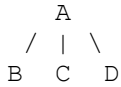
A General Tree can be converted into a Binary Tree using the **Left Child – Right Sibling Representation**.

Steps

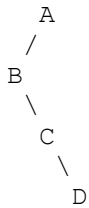
1. Connect the first child as the **Left Child**.
2. Connect all remaining children as **Right Siblings**.
3. Remove extra parent-child links.

Example

General Tree



Binary Tree



5.6 Explain the Linear Representation and Linked List Representation of a Binary Tree

A. Linear Representation (Array Representation)

Definition

A Binary Tree is stored in an **array**.

If a node is stored at index **i**:

- Left Child = $2i$
- Right Child = $2i + 1$
- Parent = $i / 2$

Example

Array Index : 1 2 3 4 5

Elements : A B C D E

Advantages

- Easy implementation.
- Fast access.

Disadvantages

- Wastage of memory for sparse trees.
- Fixed size.

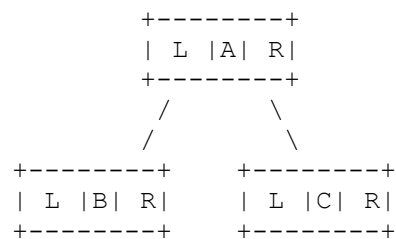
B. Linked List Representation

Definition

Each node contains:

- Left pointer
- Data
- Right pointer

Diagram



Node Representation in C

```

struct node
{
    int data;
    struct node *left;

```

```
    struct node *right;  
};
```

Advantages

- Dynamic memory allocation.
- No memory wastage.
- Easy insertion and deletion.

Disadvantages

- Extra memory for pointers.
 - More complex implementation.
-

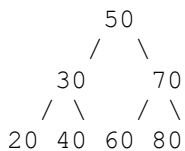
5.7 Define Binary Search Tree (BST)

Definition

A **Binary Search Tree (BST)** is a Binary Tree in which:

- All elements in the **left subtree** are smaller than the root.
- All elements in the **right subtree** are greater than the root.

Diagram



Properties

1. Left child < Root.
2. Right child > Root.
3. Both left and right subtrees are also Binary Search Trees.

Advantages

- Fast searching.
- Easy insertion.
- Easy deletion.

- Efficient sorting.

Applications

- Database indexing.
 - Dictionary implementation.
 - File management.
 - Searching applications.
-
-

5.8 Difference Between Binary Search Tree and Binary Tree

Binary Tree	Binary Search Tree (BST)
Each node has at most two children.	Each node has at most two children and follows BST property.
No ordering of elements.	Left subtree < Root < Right subtree.
Searching is slower.	Searching is faster.
Used for general hierarchical data.	Used for searching and sorting.

5.9 Perform Various Traversals on Binary Search Trees

Definition

Tree Traversal is the process of visiting every node of a tree exactly once.

Types of Traversals

1. Inorder Traversal (LNR)

Order: Left → Root → Right

Algorithm

Step 1: Traverse left subtree.
Step 2: Visit root.
Step 3: Traverse right subtree.

2. Preorder Traversal (NLR)

Order: Root $\rightarrow$ Left $\rightarrow$ Right

Algorithm

Step 1: Visit root.
Step 2: Traverse left subtree.
Step 3: Traverse right subtree.

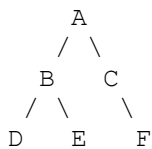
3. Postorder Traversal (LRN)

Order: Left $\rightarrow$ Right $\rightarrow$ Root

Algorithm

Step 1: Traverse left subtree.
Step 2: Traverse right subtree.
Step 3: Visit root.

Example



Traversal Output

Inorder D B E A C F

Preorder A B D E C F

Postorder D E B F C A

5.10 Construct a Binary Tree using Inorder and Preorder Traversals

Rule

- First element of **Preorder** is the **Root**.
- Find the root in **Inorder**.
- Left part forms the left subtree.
- Right part forms the right subtree.
- Repeat recursively.

Example

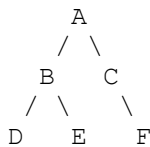
Inorder

D B E A C F

Preorder

A B D E C F

Constructed Tree



5.11 Construct a Binary Tree using Inorder and Postorder Traversals

Rule

- Last element of **Postorder** is the **Root**.
- Find the root in **Inorder**.
- Left part forms the left subtree.
- Right part forms the right subtree.
- Repeat recursively.

Example

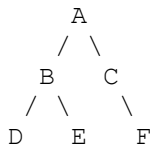
Inorder

D B E A C F

Postorder

D E B F C A

Constructed Tree



5.12 Importance of Binary Search Trees over General Trees

Advantages of BST over General Tree

1. Faster searching.
 2. Faster insertion.
 3. Faster deletion.
 4. Data remains in sorted order (Inorder Traversal).
 5. Better performance for large data.
 6. Widely used in databases and file systems.
-

5.13 Perform Insertion, Deletion, Search and Various Traversal Operations on a Binary Search Tree

1. Insertion

Algorithm

Step 1: If tree is empty, create root.
Step 2: Compare new value with root.
Step 3: If smaller, move to left subtree.

Step 4: If greater, move to right subtree.
Step 5: Insert at appropriate position.

2. Deletion

Algorithm

Step 1: Search the node.
Step 2: If node is a leaf, delete it.
Step 3: If node has one child, replace it with its child.
Step 4: If node has two children, replace it with its inorder successor or predecessor.

3. Searching

Algorithm

Step 1: Compare key with root.
Step 2: If equal, element found.
Step 3: If smaller, search left subtree.
Step 4: If greater, search right subtree.
Step 5: Repeat until found or tree becomes empty.

4. Traversals

- Inorder (LNR)
 - Preorder (NLR)
 - Postorder (LRN)
-

5.14 Explain the Program of Binary Search Tree with All Operations

Simple C Program

```
#include<stdio.h>
#include<stdlib.h>

struct node
{
    int data;
    struct node *left,*right;
};
```

```

struct node* createNode(int data)
{
    struct node *newnode;
    newnode=(struct node*)malloc(sizeof(struct node));

    newnode->data=data;
    newnode->left=NULL;
    newnode->right=NULL;

    return newnode;
}

struct node* insert(struct node *root,int data)
{
    if(root==NULL)
        return createNode(data);

    if(data<root->data)
        root->left=insert(root->left,data);
    else
        root->right=insert(root->right,data);

    return root;
}

void inorder(struct node *root)
{
    if(root!=NULL)
    {
        inorder(root->left);
        printf("%d ",root->data);
        inorder(root->right);
    }
}

int main()
{
    struct node *root=NULL;

    root=insert(root,50);
    insert(root,30);
    insert(root,70);
    insert(root,20);
    insert(root,40);

    printf("Inorder Traversal:\n");
    inorder(root);

    return 0;
}

```

Note: Similar functions can be written for **search**, **delete**, **preorder**, and **postorder** traversals.

5.15 List the Applications of Trees

Applications of Trees

1. File and Folder Management.
 2. Database Indexing.
 3. Expression Trees.
 4. Binary Search Trees.
 5. Decision Trees.
 6. Compiler Syntax Trees.
 7. Artificial Intelligence.
 8. XML and HTML Document Structure.
 9. Routing Tables in Networks.
 10. Dictionary and Spell Checker.
-

Summary of Tree Traversals

Traversal	Order
Inorder	Left → Root → Right (LNR)
Preorder	Root → Left → Right (NLR)
Postorder	Left → Right → Root (LRN)
